

Project Software Overview:

NUWC_SDP17

This document provides a high-level overview of the **NUWC_SDP17** software system. The application is designed to facilitate chip testing by coordinating configuration data, executing test sequences via serial communication with a microcontroller (MCU), and generating comprehensive result reports.

Core Architecture

The system follows an object-oriented architecture built on the **Qt framework**, utilizing C++ for core logic and QML for the user interface. The architecture is divided into four primary functional areas:

- Configuration & Parsing:** Handling YAML-based input for chip and test definitions.
- Communication:** Managing the serial interface with external hardware.
- Control Logic:** Coordinating the execution flow of tests.
- Reporting & Utilities:** Exporting data and managing user session history.

Key Components and Classes

1. Configuration Management

- YamlProcessor:** Handles the loading and parsing of YAML files to convert them into structured data used by the controller.
- ChipConfiguration (Struct):** Represents the physical metadata of a chip, including mappings for chip info, pin names, and pin configurations.
- Tests (Struct):** Stores a collection of test cases parsed from YAML, including test instructions, descriptions, and expected outputs.

2. Execution and Control

- TestController:** Coordinates the test process by interacting with the `YamlProcessor` and `SerialComms`. It supports starting, pausing, resuming, and stopping tests.
- SerialComms:** Manages data exchange with the MCU. It includes functions for calculating and verifying CRC (Cyclic Redundancy Check) to ensure data integrity during transmission.

3. Reporting and Data Export

- PdfReportGenerator:** Formats test results, chip metadata, and pass/fail statistics into a professional PDF report.

- **Exporter:** Provides functionality to export application data into **JSON** or **CSV** formats.
 - **RecentFilesManager:** Handles the persistence of recently opened files to allow users to quickly access previous work.
-

System Interactions

The software operates through a coordinated workflow between these modules:

1. **Data Loading:** The `YamlProcessor` reads a configuration file and populates the `ChipConfiguration` and `Tests` structures.
 2. **Command Flow:** The `TestController` retrieves these instructions and uses `SerialComms` to send the byte stream to the MCU.
 3. **Feedback Loop:** As the MCU responds, `SerialComms` emits a `dataReceived` signal. The `TestController` logs these messages and determines if the test passed based on the expected bytes.
 4. **Finalization:** Once the tests are complete, the user can generate a report via the `PdfReportGenerator` or export raw data using the `Exporter`.
-

File Structure Summary

File Category	Key Files	Purpose
Logic & Control	<code>testcontroller.cpp/h</code>	Main test coordination and state management.
Communication	<code>serialcomms.cpp/h</code>	MCU serial interface and CRC validation.
Data Parsing	<code>yamlprocessor.cpp/h</code>	YAML loading and data structure population.
Output	<code>pdfreportgenerator.cpp/h</code> , <code>exporter.cpp/h</code>	Document and data generation.
Entry Point	<code>main.cpp</code>	Application initialization and component registration.